第三章 SQL 注入漏洞与防护——实验分析报告

任务 1：数据库与查询条件基础说明

1.页面中的输入为什么会与数据库查询有关：

DVWA 查询功能需要展示用户信息，所有账号、ID、姓名等数据都存放在后端 MySQL 数据库中。前端用户输入的查询内容，会被后端程序接收，拼接成 SQL 语句去数据库查询数据，再把结果返回到页面，所以用户输入直接参与数据库查询。

2.一个最小业务链路是什么：

用户输入条件→后端拼接 SQL 语句→数据库执行查询→页面返回展示结果。

3.用户输入是如何进入查询条件的：

用户输入直接到了前端，把输入参数拼接到查询条件中，然后输入内容已经进入数据库查询条件，变成查询条件的一部分。

4.为什么这为 SQL 注入创造了可能性：

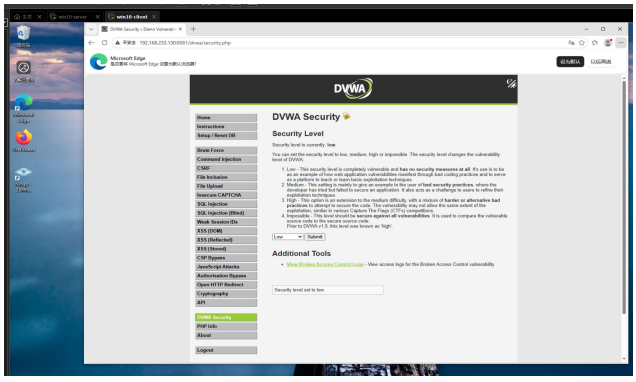
因为用户输入的内容没有被区分成“数据”和“SQL 指令”，而是直接被当作 SQL 语句的一部分解析执行。当用户输入带有 SQL 语法结构的内容（如单引号、OR、UNION 等）时，就会修改原本 SQL 语句的逻辑，让数据库执行开发者未预期的指令，从而形成 SQL 注入漏洞。

任务 2：DVWA SQL Injection Low 现象记录

访问：http://192.168.233.130:8081/dvwa

模块：SQL Injection，安全等级：Low

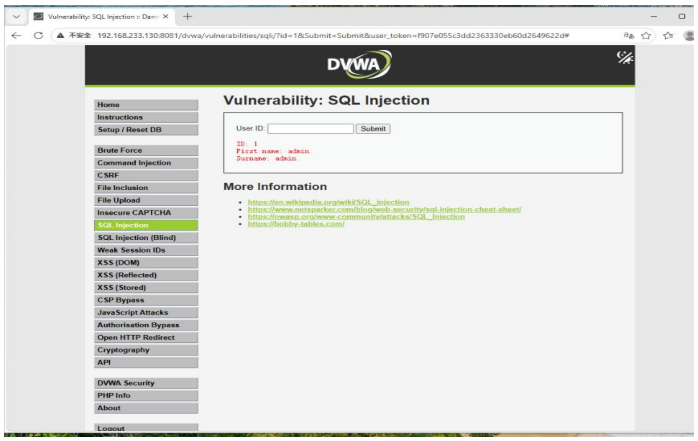
截图（选择安全等级“LOW”）：



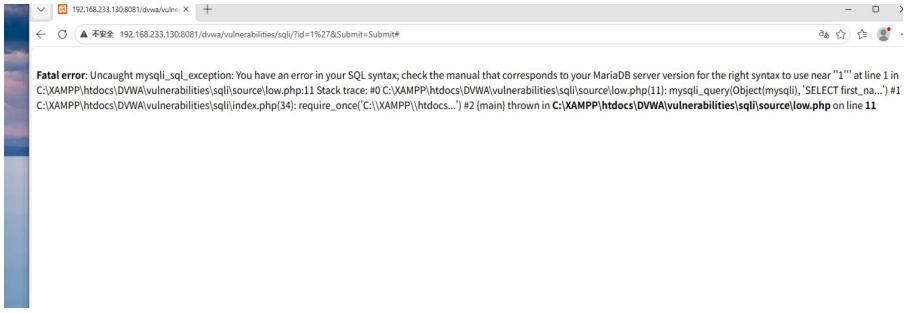
正常输入测试：

输入：1 结果：返回一条数据。正常查询，只返回 admin 一条用户信息，业务正常。

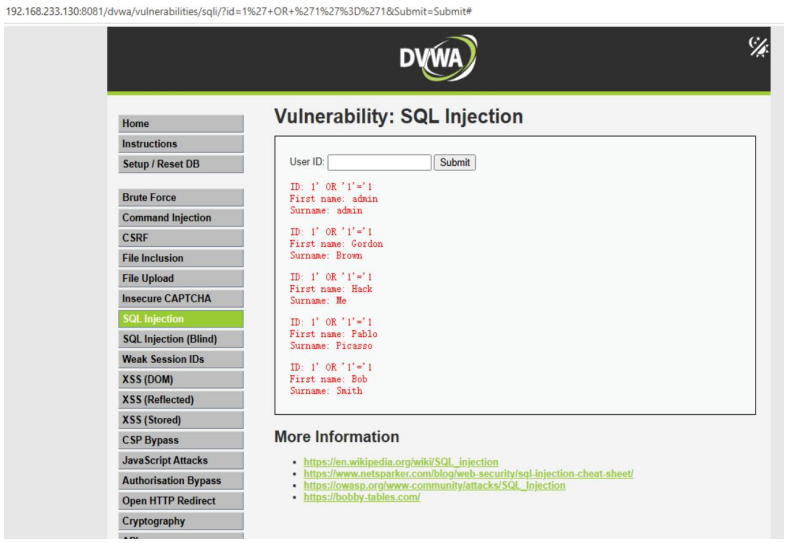
截图（输入 1 的正常返回界面）：



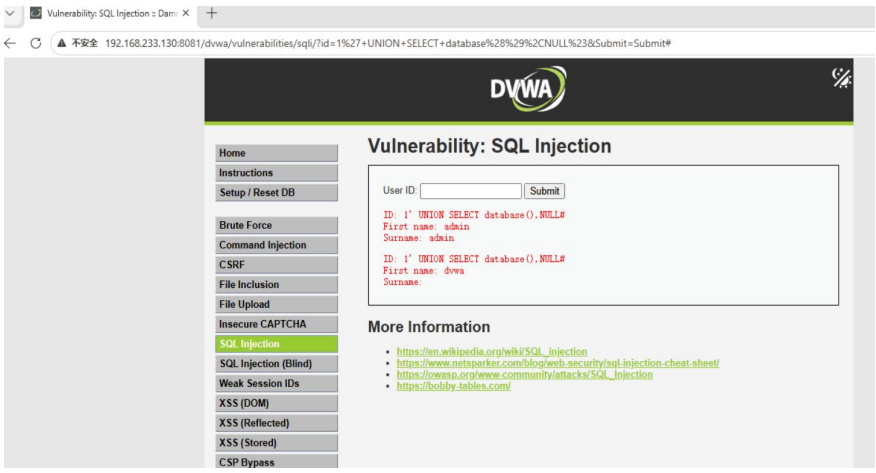
单引号测试（报错）：
输入：1'结果：页面爆出 MySQL 语法错误，说明单引号被直接代入 SQL，破坏了原有语句引号闭合，无任何过滤。
截图（输入 1'报错界面）：



万能绕过查所有用户：
输入：1' OR '1'='1 结果：返回所有用户数据
查询条件被永久成立，页面列出数据库全部用户数据，注入成功。
截图（注入成功，显示所有用户）：



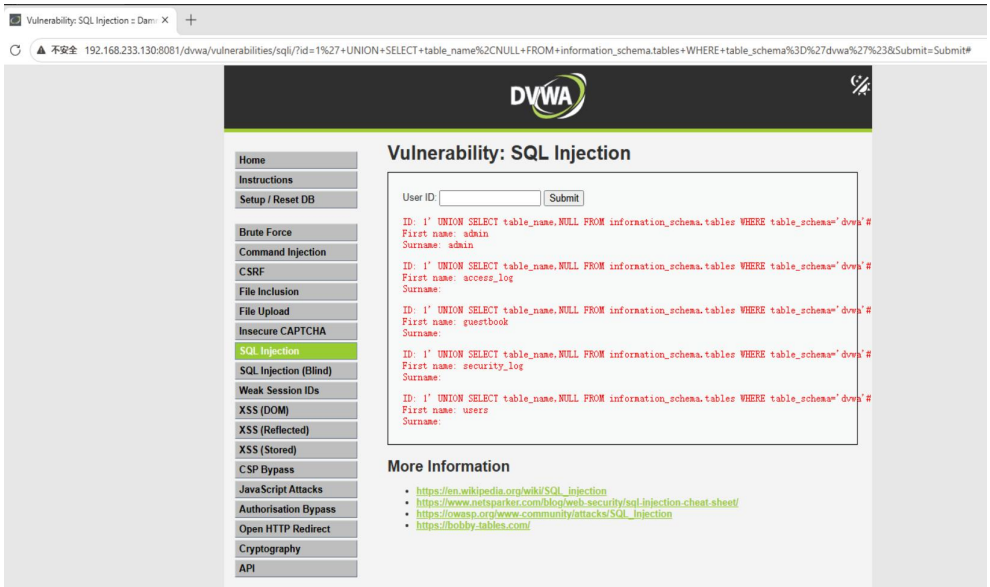
UNION 查数据库版本：
输入：1' UNION SELECT database(),NULL #
利用联合查询，读出数据库版本等敏感信息，证明可非法查询额外数据。
截图：



输入： 1' UNION SELECT table_name, NULL FROM information_schema.tables WHERE table_schema='dvwa' #

获取数据库中的所有表

截图：



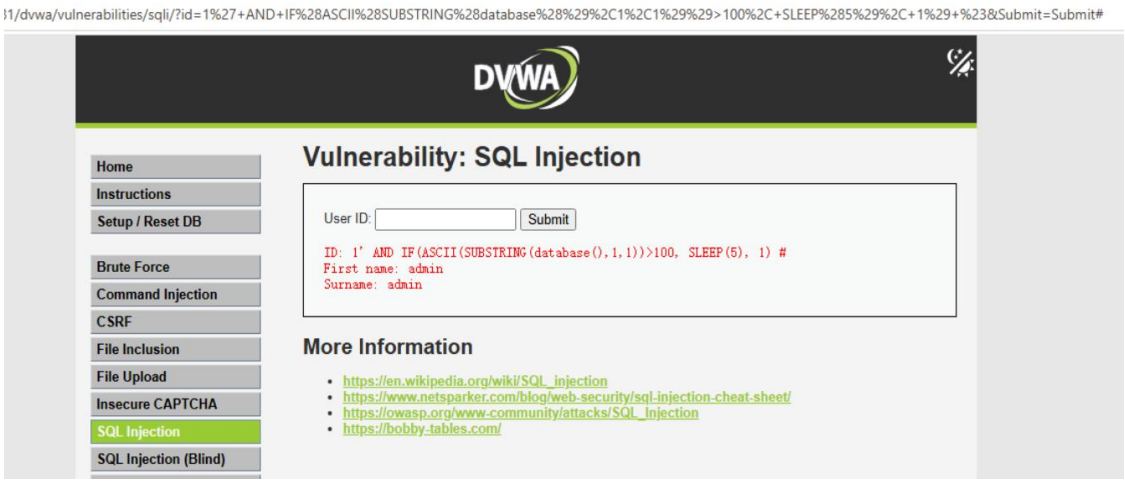
SQL 盲注：布尔盲注+时间盲注

输入：1' AND (SELECT length(database()))=4 #

截图：

输入：1' AND IF(ASCII(SUBSTRING(database(),1,1))>100, SLEEP(5), 1) #

截图：



现象总结：Low 级别完全拼接用户输入，无防护，字符型注入完全可行。

SQL 注入的原理：用户输入未经过滤，导致 SQL 语句被篡改执行。

漏洞利用方法：使用 OR1=1、UNION 查询数据库信息。

注入过程：探测注入点、构造注入、提取信息（数据库名、表、字段、用户名和密码）

任务 3：机制分析与差异比较

1.输入污染查询结构完整过程：

用户提交恶意输入→后台直接拼接 SQL→数据库按 SQL 语法解析→恶意条件生效/执行→页面返回异常数据/报错。

2.为什么根因是“结构污染”，而不是“字符本身危险”：

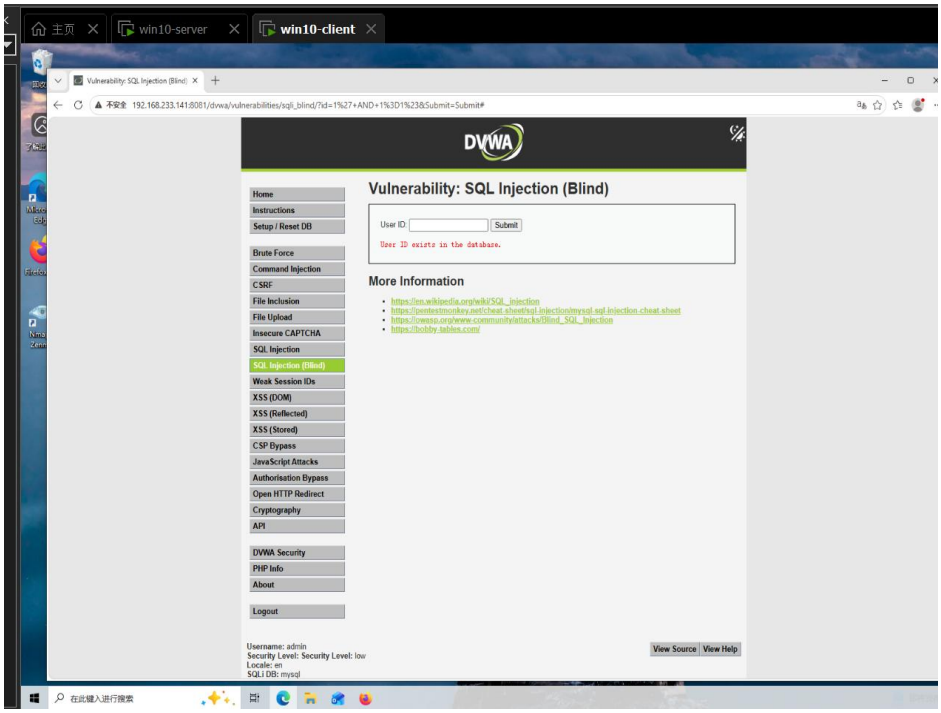
单引号、OR 本身只是普通字符，危险的是输入被当成 SQL 代码执行，修改了查询逻辑与语句结构。即使不用特殊符号，数字型注入依然可以攻击，足以说明问题在结构拼接。

3.Low 与 Medium 的差异

Low：直接将用户输入拼接到 SQL 语句中，无任何过滤，属于字符型注入，用户输入的单引号可直接修改 SQL 结构；

Medium：对单引号做了转义过滤，屏蔽字符型注入；但仍采用数字型 SQL 拼接，不用单引号的 1 OR 1=1 依旧可以注入，只是防护了表面特殊字符，没解决根本拼接问题。

如图为 Medium 级别的数字型注入成功：



差异核心：Low 是“无过滤的字符串拼接”，可直接通过单引号修改查询结构；Medium 通过转义和类型转换，限制了字符型注入，但仍存在数字型注入风险。

4.普通 SQL 注入与盲注的差异

普通 SQL 注入：页面直接回显数据、报错，能直接拿到库表内容。

盲注：关闭错误提示、无数据回显，只能通过页面空白/正常/延时，间接判断数据库内容。

5.为什么盲注和普通注入根因相同，但证据形式不同：

根因相同：两者根源都是直接拼接用户输入、未隔离代码与数据；

证据形式不同：普通注入有直接数据回显当证据；盲注关闭报错、不回显数据，只能靠页面行为差异作为判断证据。

判断题
1.SQL 注入根因选：B.输入进入了不该进入的查询结构
理由：SQL 注入的本质是用户输入被当作 SQL 指令的一部分解析，污染了原本的查询结构，而非输入中的字符本身有危险性。即使没有特殊字符，只要输入能修改查询结构，依然会产生注入漏洞。
2.普通注入与盲注最大差异：B.证据形式不同
理由：两者的根因都是查询结构被污染，只是普通注入有直接数据回显，可直接获取数据；盲注无直接回显，只能通过页面行为差异推断数据，核心差异是证据形式而非根因。

任务 4：SQL 注入防护清单与解释
1.参数化查询
解决的问题：将用户输入与 SQL 指令分离，用户输入始终被当作“数据”处理，无法修改 SQL 语句的查询结构
控制层级：根因控制（直接解决查询结构被污染的核心问题）
不能替代什么：不能替代输入校验、最小权限等措施，无法防范业务逻辑漏洞或已泄露权限的滥用
2.输入校验
解决的问题：限制用户输入的格式、长度、字符范围（如 ID 仅允许数字），拒绝异常输入
控制层级：暴露面控制（减少可被利用的输入点）
不能替代什么：不能替代参数化查询，复杂场景下过滤规则易被绕过（如双写、编码绕过）
3.错误处理
解决的问题：关闭数据库错误回显，统一返回模糊错误信息，不暴露数据库结构、语法细节
控制层级：证据控制（隐藏注入的直接证据，提升利用难度）
不能替代什么：不能防范注入本身，只是增加了盲注的难度，无法阻止注入执行
4.最小权限
解决的问题：数据库账号仅授予业务所需的最低权限（如查询用户信息的账号仅能访问 users 表，无 DROP、ALTER 等权限）
控制层级：后果缩减（限制注入成功后的影响范围）
不能替代什么：不能防范注入漏洞本身，只能降低漏洞被利用后的损失
5.ORM 框架使用
解决的问题：框架内置参数化查询逻辑，自动处理输入与 SQL 指令的分离，避免手动拼接 SQL
控制层级：根因控制（开发层面）
不能替代什么：不能替代输入校验与权限控制，不当使用原生 SQL 仍可能引入注入风险

收口项 1：SQL 注入因果链/防护关系图
核心因果链：输入→直接 SQL 拼接→查询结构被污染→解析执行→回显/行为差异→注入成功
防护对应关系：参数化查询阻断“拼接”环节（根因控制），输入校验减少暴露面，错误处理隐藏证据，最小权限缩减后果范围。

收口项 2：学生自评表
1.我本章最清楚的一点是什么？
明确了 SQL 注入的核心根因是“查询结构被污染”，而非特殊字符本身，理解了输入被拼

接进 SQL 语句的完整链路，以及不同防护措施对应的控制层级。
2.我最容易混淆的一点是什么？
容易混淆“输入过滤”和“参数化查询”的作用层级，曾误以为过滤特殊字符就能完全防范注入，忽略了结构污染的核心问题，也容易混淆盲注与普通注入的根因和证据形式差异。
3.我本次作业中哪一段最需要教师复核？
Low 与 Medium 级别的差异分析部分，以及盲注与普通注入的根因对比、防护措施的层级划分，需要确认对两者核心差异的理解是否准确。
4.如果重做一次，我会优先改哪一部分？
会补充更多 Medium 级别的实验现象记录，通过具体输入与返回差异，更直观地说明过滤措施的局限性；同时完善因果链图示的细节，让防护措施与链路的对应关系更清晰。

收口项 3：AI 输出审核与人工复核记录
1.本作业哪些内容使用过 AI 辅助？
报告的整体框架搭建、SQL 注入因果链图示生成、防护措施的层级分析、部分术语的规范表述参考了 AI 辅助整理。
2.哪些判断是你自己复核后保留的？
对 SQL 注入根因的判断、Low/Medium 级别差异的核心分析、强制判断题的结论、普通注入与盲注的差异分析，均通过 DVWA 实验验证后保留。
3.哪些内容你发现 AI 输出不准确并进行了修正？
修正了 AI 将“输入过滤” 描述为根因控制的错误，明确输入过滤属于暴露面控制，无法替代参数化查询解决结构污染问题；补充了 DVWA 实验中的具体输入与返回细节，避免泛泛而谈；修正了 AI 对盲注与普通注入根因的混淆表述，明确两者根因相同、证据形式不同。
4.你最终如何确认结论和证据是一一对应的？
每一个结论（如“输入被直接拼接进 SQL 语句”）都对应了 DVWA 实验中的具体现象（如输入 1'返回语法错误），并核对了 SQL 语句模板与现象的逻辑关系，确保结论均有实验证据支撑；同时对照提交前自查清单，确认所有要点均已覆盖。

提交前自查清单
[√]我是否只在授权环境中完成实验
[√]我的截图是否已脱敏
[√]我是否说明了“正常查询变化”和“查询逻辑被改变”的区别
[√]我是否把根因写成“结构被污染”，而不是只写“特殊字符”
[√]我是否区分了普通 SQL 注入与盲注的证据差异
[√]我是否把参数化查询、错误处理、最小权限放回不同控制层
[√]我是否完成了关系图/关系表
[√]我是否填写了 AI 输出审核与人工复核记录